Running `femm-lua` — the Lua 4 launcher

`femm-lua` is the convenience launcher for driving femm-qt with the legacy FEMM **Lua 4** scripting language. It starts the bundled Lua 4 interpreter, loads the `femm.lua` binding, opens a connection to femm-qt (launching the program if needed), and then either runs your script or drops you at an interactive Lua prompt.

There are two copies, one per platform, that behave identically:

- **`femm-lua.bat`** — Windows
- **`femm-lua.sh`** — macOS / Linux / FreeBSD

Both live in the femm-qt **`bin`** directory alongside everything they need: the `lua4` interpreter, `femm.lua`, and the `femm_gui` / `femm_headless` executables.

Short answer to "do I need it on my PATH?"

No. The launcher is fully self-contained. It figures out its own location and finds the interpreter, the binding, and the femm-qt executables *relative to itself* — it even sets `FEMM_DIR` for you (pointed at its own folder, only for the duration of the run). Nothing needs to be on your `PATH`, and you do **not** need to set any environment variable yourself.

The only thing `PATH` buys you is typing convenience. If the `bin` directory is on your `PATH`, you can type `femm-lua` from any working directory. Otherwise you just invoke it by its full path:

```
REM Windows
C:\path\to\femm-qt\build\bin\femm-lua.bat myscript.lua

# macOS / Linux / FreeBSD
/path/to/femm-qt/build/bin/femm-lua.sh myscript.lua
```

(The `FEMM_DIR` variable that the other bindings — octavefemm, pyfemm, mathfemm, juliafemm — require is a *separate* matter; see When you need `FEMM_DIR` below. The `femm-lua` launcher never needs you to set it.)

Running it day-to-day (recommended setup)

Nobody wants to type the launcher's full path every time. The *script* name is already resolved against your **current directory**, so the only awkward part is the launcher path — and the usual fix is to put the `bin` directory on your `PATH` once. Then, from whatever folder your `.lua` files live in:

```
cd C:\my\project
femm-lua myFile.lua
```

This is purely convenience: it does not change how anything is *located* (the launcher still self-finds the interpreter, binding, and executables) — it just lets you type `femm-lua` instead of the full path.

Windows — add `bin` to PATH (one time):

- *Recommended:* Start → search "environment variables" → **Edit environment variables for your account** → select **Path** → *Edit* → *New* → add `C:\path\to\femm-qt\build\bin` → OK. Open a new terminal.
- *Quick alternative:* `setx PATH "%PATH%;C:\path\to\femm-qt\build\bin"` (heads-up: `setx` truncates the stored value at 1024 characters, so prefer the GUI if your `PATH` is already long).

macOS / Linux / FreeBSD — add `bin` to PATH (one time):

Add to `~/.profile` / `~/.bashrc` / `~/.zshrc`:

```
export PATH="$PATH:/path/to/femm-qt/build/bin"
```

or symlink the launcher into a directory already on `PATH`:

```
ln -s /path/to/femm-qt/build/bin/femm-lua.sh ~/bin/femm-lua
```

Don't want to touch `PATH`? Use an alias instead, or just run the launcher by its full path (the script argument is still taken relative to your current directory either way). On Windows:

```
doskey femm-lua=C:\path\to\femm-qt\build\bin\femm-lua.bat $*
```

On Unix, a shell alias or function does the same.

Installed via the Windows installer? The installer always sets `FEMM_DIR` for you (so the GUI and the language bindings work out of the box), and it offers an optional **"Add femm-qt to the PATH"** checkbox — off by default — that performs the `PATH` step for you (and undoes it on uninstall). If you leave that unchecked, the GUI still runs from its Start-menu shortcut, and the one-time `PATH` setup above is what enables bare `femm-lua` from an arbitrary folder.

Usage

```
femm-lua [-h] [-p PORT] [script.lua [args...]]
```

Options

Option	Meaning
`-h`, `--headless`	Drive the **headless** server (no GUI window) instead of the GUI.
`-p`, `--port` *PORT*	Use TCP port *PORT* instead of the default 48015. Needed to run two or more at the same time — see Running several at once.
`-?`, `--help`	Print a usage summary and exit.
(none)	With no script argument, start an **interactive** Lua prompt.

> **Watch out:** `-h` means **headless, not help.** To get help, use `-?` or `--help`.

Options are only recognized when they come **before** the script name. Once the first non-option argument is seen, it and everything after it are treated as the script and its arguments. So `femm-lua -h sim.lua` runs `sim.lua` headless, but `femm-lua sim.lua -h` passes `-h` *to your script*.

The four ways to invoke it

1. Interactive prompt against the GUI

```
femm-lua
```

Launches `femm_gui`, connects to it, prints the Lua 4 banner, and leaves you at a `>` prompt. Type Lua commands and they drive the GUI live:

```
> newdocument(0)
> mi_drawrectangle(0, 0, 10, 10)
> mi_zoomnatural()
```

Exit the prompt (`Ctrl+Z` then Enter on Windows, `Ctrl+D` on macOS/Linux, or `quit()`); the femm-qt window closes with it (see Session lifetime).

2. Interactive prompt against the headless server

```
femm-lua -h
```

Same interactive prompt, but backed by `femm_headless` (`femm_headlessw` on Windows) — no GUI window appears. Useful for poking at the API or batch-style exploration on a machine with no display.

3. Run a script against the GUI

```
femm-lua myscript.lua
```

Launches the GUI, connects, then runs `myscript.lua`. You see the geometry, mesh, and results appear in the window as the script executes.

4. Run a script headless

```
femm-lua -h myscript.lua
```

Runs the script with no window — the normal choice for automation, solving on a server, or CI.

Passing arguments to your script Anything after the script name is handed to the script (Lua 4 exposes them in the global `arg` table):

```
femm-lua -h sweep.lua 0.5 2.0 40
```

The script path itself is resolved relative to your **current working directory**, not the `bin` folder — so keep your `.lua` files wherever you like and run the launcher from there (or give a full path to the script).

What the launcher does for you

Internally the launcher runs the bundled interpreter with a small preamble prepended:

```
lua4  -e "dofile([[<bin>/femm.lua]]); openfemm(<mode>)"  -f <yourscript> <args...>
```

where `<mode>` is `0` for the GUI or `1` for headless. In other words, **it calls `openfemm()` for you before your script (or the prompt) starts.** Two consequences:

- Your script can issue FEMM commands (`newdocument`, `mi_*`, `mo_*`, ...) immediately — the connection is already open.
- **Your script should not call `openfemm()` again.** femm-qt accepts only one client at a time, so a second `openfemm()` from inside a launcher-run script hits the single-client guard and fails with `error:femm-qt is already connected to another client`. If you are adapting a classic FEMM script that begins with `openfemm()` (and ends with `closefemm()`), remove those two calls — the launcher handles them.

How `openfemm()` finds and starts femm-qt

1. Try to connect to a femm-qt already listening on `127.0.0.1:48015`. If one is there, latch onto it (whether it's a GUI or headless instance).
2. If nothing answers, spawn the appropriate executable from `FEMM_DIR` (which the launcher has set to the `bin` folder) as `"<exe>" --port 48015`, in the background.
3. Poll for the server's socket to come up (up to ~20 s, to cover a cold first start), then connect.

If the executable can't be found you get `Cannot find <exe>. Set FEMM_DIR environment variable.` — which under the launcher should never happen, because the binaries sit right next to it.

Running several at once

One femm-qt serves **one client on one port**. The default port is 48015, so two `femm-lua` runs started at the same time both try to use it: both find nothing listening, both launch a server, and only one of them can bind the port. The other is turned away.

Sequential runs in a batch file are fine — each one finishes and its femm-qt exits before the next starts. It is only *simultaneous* runs that collide.

Give each run its own port with `-p`:

```
start "" femm-lua -h -p 48016 job1.lua
start "" femm-lua -h -p 48017 job2.lua
start "" femm-lua -h -p 48018 job3.lua
```

```
femm-lua -h -p 48016 job1.lua &
femm-lua -h -p 48017 job2.lua &
femm-lua -h -p 48018 job3.lua &
wait
```

Any free port works; the numbers above are simply the default plus one, two and three. Each run gets its own femm-qt, and they do not see each other's documents.

If two do end up sharing a port, the one that loses says so and stops at `openfemm()`, before running any of your script:

```
error: cannot use FEMM on port 48015: femm-qt is already connected to
another client
Each FEMM running at the same time needs its own port.
Start this one with:  femm-lua -p <port> ...  (e.g. -p 48016)
```

The port must be chosen on the **command line**, not inside the script: the launcher connects before your first line runs.

Session lifetime

The femm-qt instance is tied to your Lua session. When the connection closes — you exit the interactive prompt, or the script finishes, or `lua4` otherwise exits — femm-qt sees the disconnect and **shuts itself down** (it calls `exit(0)`), so you're never left with an orphaned GUI or headless process. This is true even for a GUI you latched onto: disconnecting ends it. If you want the window to stay up after a script, run the script from the interactive prompt with `dofile("myscript.lua")` instead, and quit the prompt when you're done.

When you actually need `FEMM_DIR`

You do **not** need `FEMM_DIR` for the `femm-lua` launcher — it sets it itself. You need it only when something *else* has to locate the femm-qt executables:

- The other language bindings (octavefemm, pyfemm, mathfemm, juliafemm, scifemm) — their `openfemm`/`open_femm` look up `FEMM_DIR`.
- Driving the binding by hand without the launcher, e.g. `lua4 -e "dofile('femm.lua'); openfemm()"`.

In those cases point `FEMM_DIR` at the `bin` directory:

```
# macOS / Linux / FreeBSD
export FEMM_DIR=/path/to/femm-qt/build/bin
```

```
REM Windows
set FEMM_DIR=C:\path\to\femm-qt\build\bin
```

macOS, from an installed `.app`. A distributed femm-qt has no `bin/` directory – everything lives inside the bundle – so point `FEMM_DIR` at the bundle's executable folder instead:

```
export FEMM_DIR=/Applications/femm_gui.app/Contents/MacOS
```

The binding files themselves ship **inside** the bundle on macOS, since a distributed `.app` has no sibling folders to put them in:

```
/Applications/femm_gui.app/Contents/Resources/bindings/luafemm
```

The examples are alongside them, in `Contents/Resources/examples/` – but those are read-only **masters**. Use the `Examples` folder the DMG carries next to the app: drag it somewhere of your own and work there. Example scripts need that, not just convenience: the FEMM idiom is `open("model.fem")` followed by `mi_saveas("temp.fem")` so the original is never modified, and both paths are relative to the script's own directory. Inside the read-only bundle the save fails and the script stops.

Either setting works from a source build. The bundle path is worth preferring even there: `femm_headless` has no `.app` form, so with `FEMM_DIR` set to `build/bin` the bindings launch the loose copy in `bin/` while the GUI comes from the bundle. Both find the material libraries – on macOS FEMM keeps the writable copies in `~/Library/Application Support/FEMM/`, shared by every femm-qt process on the machine – but pointing at the bundle keeps one binary location rather than two.

Troubleshooting

`femm-lua: cannot find lua4 next to this script` (or `femm.lua`) — the launcher was moved away from the `bin` directory. It must stay beside `lua4`, `femm.lua`, and the femm-qt executables. Run it from where the build placed it (or copy the whole set together).

`cannot use FEMM on port 48015: femm-qt is already connected to another client` — something else is already connected to femm-qt on that port: another binding session, a second `femm-lua` started at the same time, or your own script calling `openfemm()` twice. Give this run its own port with `-p` (see Running several at once), close the other client, or remove the redundant `openfemm()`.

Before this was detected at connect time, the same situation surfaced one command later as `FEMM connection closed unexpectedly`, which pointed nowhere useful.

`femm-qt launched but TCP connection failed (timed out)` — the executable started but never began listening within ~20 s. Usually a blocked port 48015, or a security prompt/antivirus stalling the first launch. Check that `femm_gui/femm_headless` can run directly.

The GUI flashes and closes immediately after a script — expected: the session ended, so femm-qt exited. See Session lifetime.